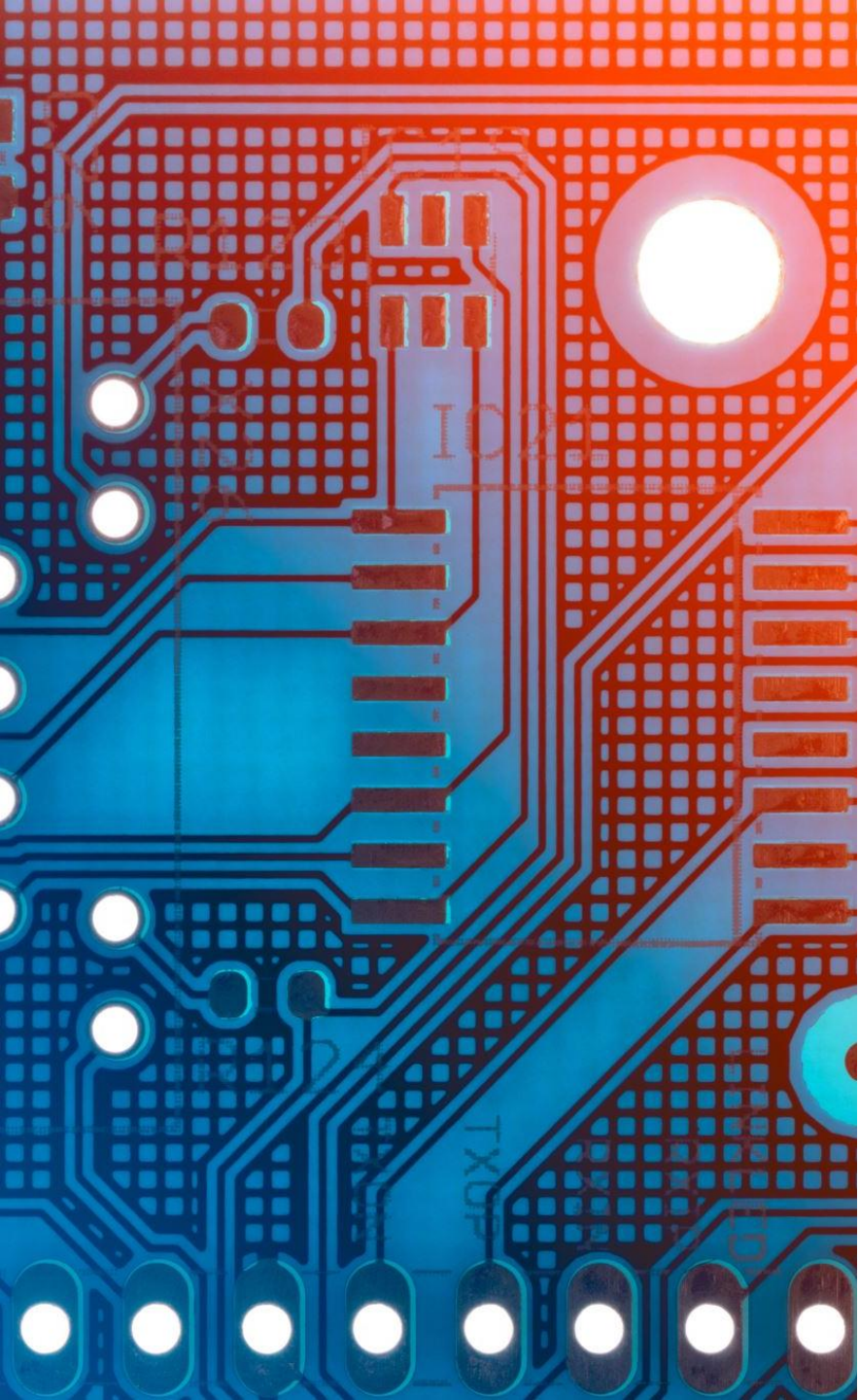




Hardware-Specific Optimizations - Microcontrollers

Microcontroller Optimizations

Objective: Minimize memory and compute usage for resource-constrained devices (e.g., [ARM Cortex-M](#)).

Key Considerations:

- Limited RAM/Flash memory (e.g., KBs of memory).
- No OS or dynamic memory allocators; avoid heap usage.
- Cache misses are costly; optimize access patterns.

Small Data Types:

- Use 8-bit integers (int8) or binary weights instead of FP32 to save memory.
- Quantization is critical (e.g., TensorFlow Lite for Microcontrollers).

Cache-Friendly Access:

- Use linear or sequential memory access to avoid cache thrashing.
- Avoid random jumps (e.g., linked lists).

Static Memory Allocation:

- Pre-allocate buffers during compilation (no malloc at runtime).

Example:

- Replace a float32 dense layer with a int8 quantized layer, reducing memory by 4x.

Model Architecture Optimization

1. FLOP Estimation

Dense (Fully Connected)

- $\text{FLOPs} \approx 2 \times (\text{input_size} \times \text{output_size})$
- Example: $1024 \rightarrow 512 \Rightarrow 2 \times 1024 \times 512 = 1.05\text{M FLOPs}$

Convolution (standard, no bias)

- $\text{FLOPs} \approx 2 \times (k_h \times k_w \times \text{in_ch}) \times (\text{out_h} \times \text{out_w}) \times \text{out_ch}$
- Example: 3×3 , $\text{in_ch}=64$, $\text{out_ch}=128$, $\text{output}=56 \times 56$
 - $2 \times 3 \times 3 \times 64 \times 56 \times 56 \times 128 \approx 11.1\text{G FLOPs}$

Notes

- “2×” counts multiply plus add.
- Grouped/depthwise: divide by groups; depthwise $\Rightarrow$ groups=in_ch, out_ch=in_ch.
- FLOPs help rank hotspots; also consider memory and bandwidth.

Model Architecture Optimization

2. Architecture Comparison

Model A: 100M FLOPs → 1000ms on target hardware

Model B: 50M FLOPs → expect ~500ms

Model C: 200M FLOPs → expect ~2000ms

Key Insight: FLOPs scale **linearly with latency** (assuming similar hardware utilization).

- Halving FLOPs (Model B) ≈ halves latency.
- Doubling FLOPs (Model C) ≈ doubles latency.

Purpose: Compare models by **FLOPs vs. accuracy**.

- Loads candidate models (e.g., different CNN architectures).
- Computes total FLOPs and evaluates accuracy.
- Outputs a trade-off analysis (e.g., "Is 2× FLOPs worth +1% accuracy?").

Model Architecture Optimization

3. Optimization Strategies

1. Remove Layers with Highest FLOP/Accuracy Ratio

Example: Removing a 1M-FLOP dense layer that improves accuracy by only 0.1%.

2. Replace Expensive Operations with Efficient Alternatives

- **Examples:**

- Replace 3×3 convolutions with 1×1 convolutions (9× fewer FLOPs).
- Use **grouped convolutions** (e.g., ResNeXt) to reduce connectivity.

3. Use Depthwise Separable Convolutions

Splits a standard convolution into:

- **Depthwise Convolution:** Filters per input channel ($\text{kernel_h} \times \text{kernel_w} \times \text{input_channels}$).
- **Pointwise Convolution:** 1×1 conv to mix channels ($\text{input_channels} \times \text{output_channels}$).

FLOP Reduction:

- Standard conv: $K \times K \times C_{in} \times H \times W \times C_{out} \times 2$.
- Depthwise separable: $(K \times K \times C_{in} \times H \times W \times 1 \times 2) + (1 \times 1 \times C_{in} \times H \times W \times C_{out} \times 2)$.
- Example: For $C_{in}=32$, $C_{out}=64$, $K=3$, this reduces FLOPs by ~8–9×.

RAM Usage Analysis

```
// Static allocation approach
uint8_t tensor_arena[50 * 1024]; // 50KB arena

tflite::MicroInterpreter interpreter(
    model, resolver, tensor_arena,
    sizeof(tensor_arena), error_reporter);

// Check if arena is sufficient
if (interpreter.AllocateTensors() != kTfLiteOk) {
    // Arena too small - increase size
}
```

Dynamic Memory Requirements

TensorFlow Lite Memory Arena:

The arena holds temporary activation tensors during inference

Arena size calculation:

- `arena_size = max_simultaneous_tensor_memory`

Typical requirements by model type:

- **Simple_dense_model**: 5-20KB
- **Small_CNN**: 20-100KB
- **Efficient_transformer**: 50-200KB

Application Memory:

- Input/output buffers
- Preprocessing workspace
- Communication buffers
- Business logic variables

Edge and Microcontroller AI

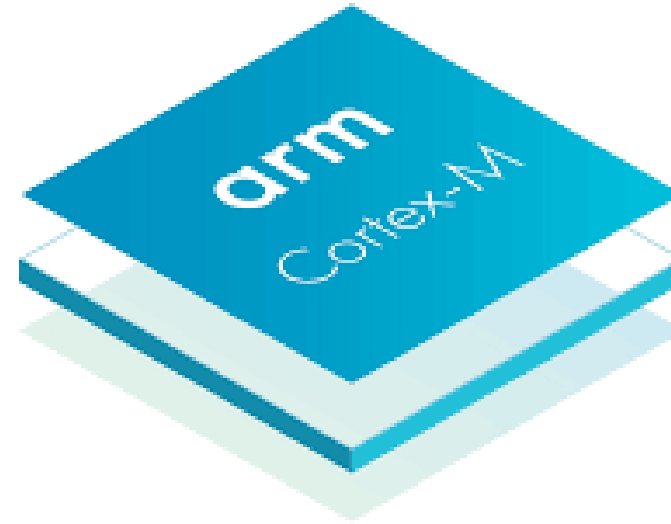
Ultra-Low Power AI Processing

Microcontroller Characteristics:

```
// Typical microcontroller AI inference
void run_inference() {
    // Model stored in flash memory (read-only)
    const int8_t* weights = (int8_t*)FLASH_MODEL_BASE;

    // Activations in SRAM (limited to ~256KB)
    int8_t activations[MAX_ACTIVATION_SIZE];

    // Quantized computation
    for (int layer = 0; layer < num_layers; layer++) {
        quantized_conv2d(&activations[input_offset],
                        &weights[weight_offset],
                        &activations[output_offset]);
    }
}
```



Hardware Constraints:

- **Memory:** 32KB - 1MB SRAM, 256KB - 2MB Flash
- **Computation:** 100 MHz ARM Cortex-M processors
- **Power:** 1-100 mW operation, battery life measured in months
- **Cost:** \$1-10 per processor in volume

Optimization Strategies:

- **8-bit quantization:** Essential for memory and computation efficiency
- **Model compression:** Pruning, knowledge distillation
- **Streaming computation:** Process data without storing full tensors
- **Hardware acceleration:** ARM CMSIS-NN optimized kernels



Architecture Selection Principles

For Mobile/Edge SoCs (Efficiency-First)

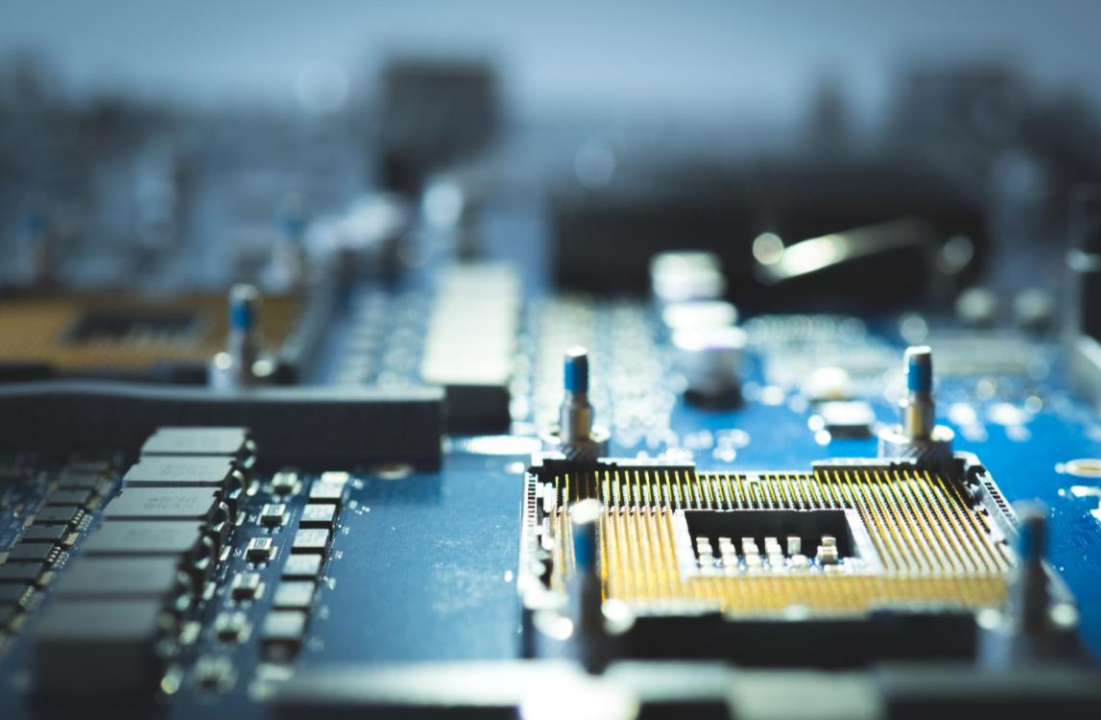
- Prioritize depthwise-separable convolutions, inverted residuals, and squeeze-and-excitation with careful channel scaling to reduce MACs and bandwidth.
 - Choose input resolutions and channel widths that fit on-chip memory to minimize DRAM traffic; cap peak activation sizes.
 - Constrain the operator set to what the device delegate accelerates (Hexagon/QNN, NNAPI, Core ML, CUDA/TensorRT) to avoid falling back to CPU.
 - Optimize for energy: use quantization (INT8), layer fusion, and DVFS-friendly throughput; target steady 20–30 fps at modest clocks rather than peak bursts.
-



Architecture Selection Principles

For Microcontrollers (Memory-Constrained)

- Design with fixed memory budgets: model parameters must fit in flash; the largest intermediate tensor set must fit in SRAM.
 - Use INT8 quantization end-to-end; prefer operator patterns available in micro inference libraries (e.g., TFLite Micro).
 - Minimize simultaneous live activations: structure blocks to reuse buffers and avoid long skip connections that force multiple feature maps to reside in SRAM.
 - Keep kernels small and channels modest; rely on pruning and knowledge distillation to recover accuracy within strict size limits.
-



Workflow

- Check supported precisions on the target.
- Start with INT8 (mobile), FP16/INT8 (server), tighter bits only if necessary (MCU).
- Calibrate, test on device, iterate.

Quantization Strategy Selection

Goal: Pick the lowest precision your hardware supports without breaking accuracy.

Server GPUs

- Train: FP32 or mixed FP16.
- Infer: FP16/TF32/INT8 on tensor cores.
- Use higher precision if accuracy is critical.

Mobile Processors

- Default to INT8 for weights and activations.
- Use per-channel weights; keep 32-bit accumulators.
- Verify kernels run on the device accelerator, not CPU.

Microcontrollers

- Go lower if needed: INT8 → 4-bit weights where supported.
- Consider ultra-low-bit activations only when memory forces it.
- Ensure weights fit in flash and peak activations in SRAM.
- Quantization-Aware Training
- Use QAT for aggressive quantization or MCU targets to recover accuracy.

Memory-Constrained Model Design

Goal: Fit models in limited flash/SRAM while keeping accuracy usable

Budget Planning

- Know flash size (weights/code) and SRAM size (activations/scratch)
- Track peak live activations, not just parameters
- Use static memory plans; preallocate reuse buffers

Design Techniques

- Layer-wise processing
- Run one layer at a time
- Stream weights from flash
- Reuse a single activation buffer across layers

In-place ops

- Use ReLU and similar in-place to avoid extra tensors
- Minimize temporary buffers

Operator fusion

- Fuse Conv + BN + ReLU
- Reduce intermediate tensors and memory traffic

Quantization

- INT8 weights/activations; 32-bit accumulators if needed

Pruning and sparsity

- Remove low-importance channels/connections
- Exploit sparse formats when supported

Shape control

- Cap feature-map sizes (channels/resolution)
- Avoid long skip connections that keep tensors live

Validation

- Measure peak SRAM during inference on device
- Iterate until within flash/SRAM limits